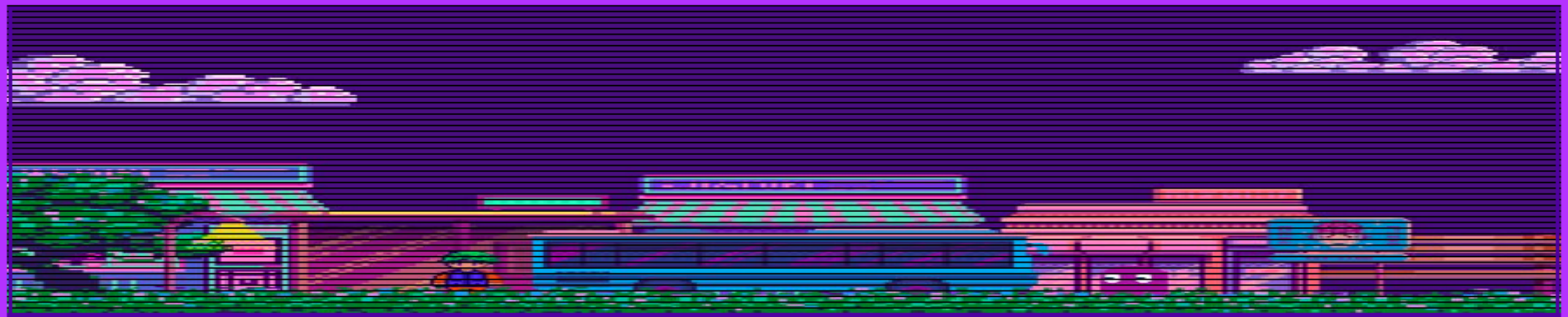
# ARCADE VAULT

## A MULTI-GAME INTERACTIVE PLATFORM USING C PROGRAMMING LANGUAGE



# ARCADE VAULT

◆ TEAM ◆

## ✦ TEAM MEMBERS ✦

01 ▶ NOEL ROBIN NINAN

02 ▶ ADITYA GOYAL

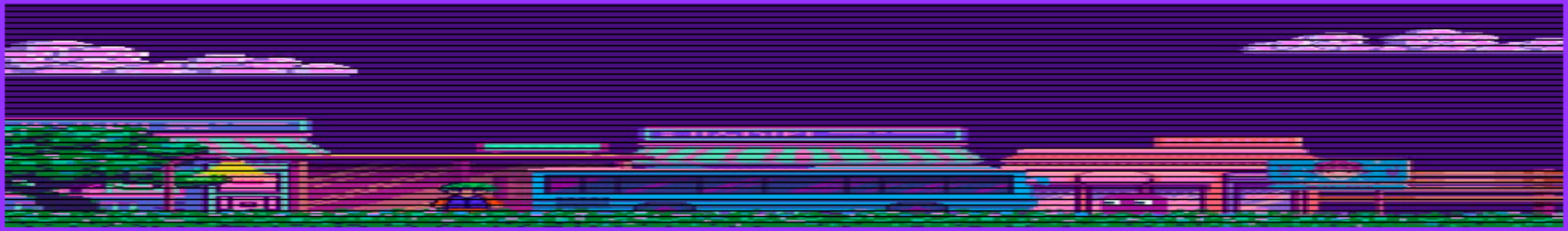
03 ▶ ARNAV LOKHANDE

04 ▶ RITESH POOJARI

PRESS START ● ● ●

4 PLAYERS READY

LET'S PLAY THE
GAME
START



# Engineered for Engagement: The Multi-Game Platform

Dive into our modular multi-game platform, designed for instant fun and cognitive stimulation through robust system architecture.

### Wordle

Challenge your vocabulary with daily word puzzles.

### Tic Tac Toe

Relive the timeless classic for quick, engaging matches.

### Math Survival

Sharpen mental math skills in a race against time.

### Card Bet Game

Wager coins on Odd or Even card draws. Press your luck!

---

The core menu leverages a persistent while loop, ensuring continuous operation until an explicit exit command. User input via scanf drives navigation. A switch statement dispatches control to Wordle, Tic Tac Toe, Math Survival, and the new Card Bet Game — which introduces rand() card draws and modulo arithmetic for odd/even betting, reinforcing loops, conditionals, and game-state management in C.

# Main Menu Code

✦ UPDATED

SLIDE 02

## Snippet 1 — Includes & Prototypes

games.c — ARCADE VAULT

games.c

```c
/* Standard input/output library
   Provides printf, scanf, etc. */
#include <stdio.h>
/* Standard library
   Provides rand(), srand(), malloc(), etc. */
#include <stdlib.h>
/* String handling library
   Provides strlen(), strcmp(), etc. */
#include <string.h>
/* Time library
   Provides time(), difftime(), time_t */
#include <time.h>
/* Function prototypes so main() knows these functions exist */
void card_bet_game();   // Card Bet Game - new addition
void wordle_game();     // Wordle Game
void tic_tac_toe();     // Tic Tac Toe with Smart CPU
void math_game();       // Math Survival Mode
/* ==================== MAIN MENU ==================== */
int main() {
    int choice;   // stores user menu choice
    /* Loop keeps running until user chooses Exit (5) */
    do {
        printf("\n===== ARCADE VAULT =====\n");
        printf("1. Card Bet Game\n");   // new game added
        printf("2. Wordle Game\n");
        printf("3. Tic Tac Toe\n");
        printf("4. Math Game\n");
```

UTF-8 | CRLF | games.c

## Snippet 2 — main() & switch

games.c — ARCADE VAULT

games.c

```c
        printf("5. Exit\n");
        printf("Enter your choice: ");
        /* Read menu choice from user */
        scanf("%d", &choice);
        /* Switch executes the selected game */
        switch (choice) {
            case 1:
                card_bet_game();   // start Card Bet Game
                break;
            case 2:
                wordle_game();     // start Wordle
                break;
            case 3:
                tic_tac_toe();     // start Tic Tac Toe
                break;
            case 4:
                math_game();       // start Math Survival
                break;
            case 5:
                printf("Exiting program. Goodbye!\n");
                break;
            default:
                printf("Invalid choice. Try again.\n");
        }
    } while (choice != 5);   // repeat until Exit chosen
    return 0;   // program ended successfully
}
```

UTF-8 | CRLF | games.c



© 2025

# Card Bet Game

SLIDE 05

## Screenshot 1

card_bet_game.c

```c
/* ================== CARD BET GAME ================== */
/* drawCard() - generates a random card value from 1 to 13 */
int drawCard() {
    return (rand() % 13) + 1;   // rand() % 13 gives 0-12, +1 shifts to 1-13
}
/* printCard() - prints the card name instead of just a number */
void printCard(int card) {
    if      (card == 1)  printf("Ace");    // 1 is displayed as Ace
    else if (card == 11) printf("Jack");   // 11 is displayed as Jack
    else if (card == 12) printf("Queen");  // 12 is displayed as Queen
    else if (card == 13) printf("King");   // 13 is displayed as King
    else                 printf("%d", card); // 2-10 printed as numbers
}
void card_bet_game() {
    srand(time(NULL));   // seed random number generator with current time for true randomness
    int coins = 50;      // player starts with 50 coins
    char choice;         // stores player's menu/guess input (o/e/y/n)
    printf("\n--- CARD BET GAME ---\n");                    // game title header
    printf("Bet on whether the card will be ODD or EVEN!\n\n"); // instructions
    while (coins > 0) {   // keep playing as long as player has coins
        /* Step 1: Show coins and place a bet */
```

main.c — C | UTF-8 | CRLF

## Screenshot 2

card_bet_game.c

```c
        int bet = 0;                    // initialize bet to 0 each round
        printf("Coins: %d\n", coins);       // show current coin balance
        printf("Enter bet: ");              // prompt player to enter bet
        scanf("%d", &bet);                  // read the bet amount from keyboard
        /* Validate the bet */
        if (bet <= 0 || bet > coins) {           // bet must be positive and within balance
            printf("Invalid bet! Try again.\n\n"); // reject invalid bet
            continue;                            // restart loop, don't proceed
        }
        /* Step 2: Guess odd or even */
        printf("Odd or Even? (o/e): ");   // ask player to guess
        scanf(" %c", &choice);           // read single character; space before %c skips leftc
        /* Step 3: Draw a card */
        int card = drawCard();   // call drawCard() to get a random card value 1-13
        printf("Card drawn: ");  // label before card name
        printCard(card);         // print the card name (Ace, Jack, Queen, King, or number)
        printf(" (%d)\n", card); // also print the raw number in brackets for clarity
        /* Step 4: Check result using modulo */
        int isEven = (card % 2 == 0);  // card % 2 == 0 means even, stores 1(true) or 0(false)
        if ((choice == 'e' && isEven) || (choice == 'o' && !isEven)) {
            // player wins if: guessed 'e' and card is even  OR  guessed 'o' and card is odd
```

main.c — C | UTF-8 | CRLF

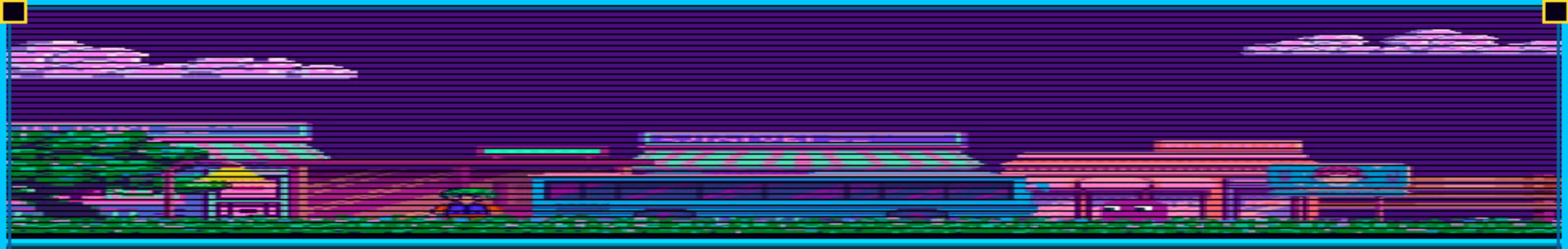
## Screenshot 3

card_bet_game.c

```c
            coins += bet;                                          // add bet to coins
            printf("Correct! You win %d coins! Total: %d\n\n", bet, coins); // show win message
        } else {
            // player loses if their guess didn't match the card
            coins -= bet;                                          // subtract bet from co
            printf("Wrong! You lose %d coins. Total: %d\n\n", bet, coins); // show lose message
        }
        /* Step 5: Check if player is out of coins */
        if (coins <= 0) {                          // if coins dropped to 0 or below
            printf("Out of coins! Game over.\n");   // game over message
            break;                                  // exit the while loop immediately
        }
        /* Step 6: Ask to play again */
        printf("Play again? (y/n): ");  // prompt to continue or quit
        scanf(" %c", &choice);          // read y or n
        if (choice == 'n') break;       // if player types n, exit the loop
        printf("\n");                   // blank line for spacing before next round
    }
    printf("Final coins: %d\n", coins); // display final coin count when game ends
}
```

main.c — C | UTF-8 | CRLF


© 2025



# Wordle Game

SLIDE 06

## Screenshot 1

```c
/* ==================== WORDLE GAME ==================== */
void wordle_game() {

    /* Array of valid 5-letter words */
    const char *words[] = {
        "apple", "grape", "mango", "peach", "berry",
        "lemon", "melon", "chili", "bread", "cheese"
    };

    int word_count = 10;  // total number of words

    /* Seed random number generator using current time */
    srand(time(NULL));

    /* Randomly select one word from the list */
    const char *secret = words[rand() % word_count];

    char guess[6];   // user guess (+1 for null terminator)
    int attempts = 6; // max number of guesses
    int won = 0;      // flag to track win condition

    printf("\n--- WORDLE GAME ---\n");
    printf("Guess the 5-letter word\n");
    printf("🟩 Correct spot | 🟧 Wrong spot | ⬜ Not in word\n\n");

    /* Loop until attempts run out */
    while (attempts > 0) {
        printf("Attempts left %d: ", attempts);

        /* Read at most 5 characters to prevent overflow */
        scanf("%5s", guess);

        /* Validate guess length */
        if (strlen(guess) != 5) {
            printf("Word must be exactly 5 letters.\n\n");
            continue; // retry without losing an attempt
        }
```

## Screenshot 2

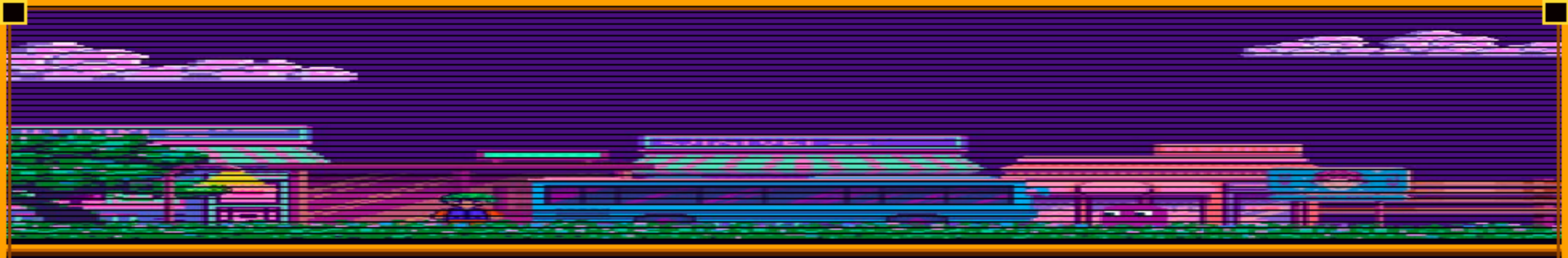
```c
        /* If guess matches secret exactly, player wins */
        if (strcmp(guess, secret) == 0) {
            printf("🟩 🟩 🟩 🟩 🟩\n");
            printf("You won 🎉\n");
            won = 1;
            break;
        }

        int used[5] = {0};  // tracks letters already matched
        char feedback[5] = {'⬜','⬜','⬜','⬜','⬜'}; // default feedback

        /* First pass: correct letter in correct position */
        for (int i = 0; i < 5; i++) {
            if (guess[i] == secret[i]) {
                feedback[i] = '🟩';
                used[i] = 1;
            }
        }

        /* Second pass: correct letter in wrong position */
        for (int i = 0; i < 5; i++) {
            if (feedback[i] == '🟩') continue;

            for (int j = 0; j < 5; j++) {
                if (!used[j] && guess[i] == secret[j]) {
                    feedback[i] = '🟧';
                    used[j] = 1;
                    break;
                }
            }
        }

        /* Print feedback for this guess */
        for (int i = 0; i < 5; i++)
            printf("%c ", feedback[i]);
        printf("\n\n");
```

## Screenshot 3

```c
        attempts--;  // consume an attempt
    }

    /* If loop ended without winning */
    if (!won)
        printf("You lost! The word was '%s'\n", secret);
}
```



© 2025

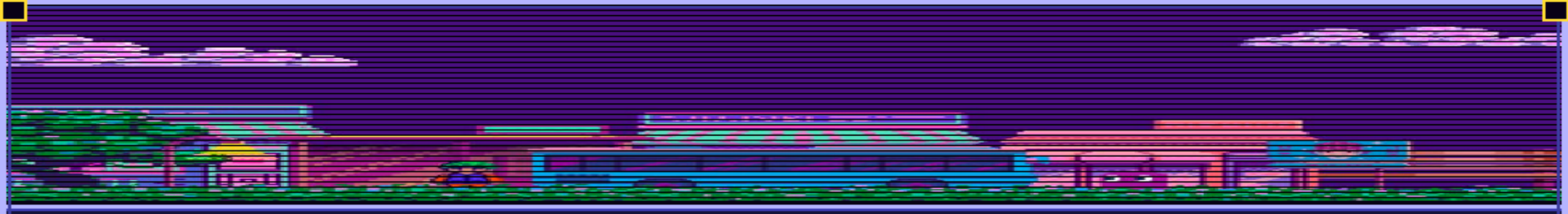


# Math Survival

SLIDE 07

12×4=? 5s TIMER

```c
/* ===================== MATH SURVIVAL GAME ===================== */
void math_game() {
    int level = 1;
    int score = 0;
    const int TIME_LIMIT = 5;

    srand(time(NULL));

    printf("\n--- MATH SURVIVAL MODE ---\n");
    printf("Answer correctly to level up.\n");
    printf("You have %d seconds per question.\n", TIME_LIMIT);
    printf("One mistake or timeout = GAME OVER.\n\n");

    while (1) {
        int max_num;
        char op;
        int a, b, correct, user;
        time_t start, end;

        /* Scale difficulty by level */
        if (level <= 3) {
            max_num = 10;
            op = (rand() % 2 == 0) ? '+' : '-';
        } else if (level <= 6) {
            max_num = 20;
            int r = rand() % 3;
            op = (r == 0) ? '+' : (r == 1) ? '-' : '*';
        } else {
            max_num = 50;
            int r = rand() % 4;
            op = (r == 0) ? '+' : (r == 1) ? '-' : (r == 2) ? '*' : '/';
        }

        a = rand() % max_num + 1;
        b = rand() % max_num + 1;

        /* Ensure division has integer result */
        if (op == '/') {
            correct = a;
            a = a * b;
        } else if (op == '+') {
            correct = a + b;
        } else if (op == '-') {
            correct = a - b;
        } else {
            correct = a * b;
        }

        printf("Level %d | Score %d\n", level, score);
        printf("Solve: %d %c %d = ", a, op, b);

        start = time(NULL);
        scanf("%d", &user);
        end = time(NULL);

        if (difftime(end, start) > TIME_LIMIT) {
            printf("\n⏰ Time's up!\n");
            break;
        }

        if (user != correct) {
            printf("\n❌ Wrong answer!\n");
            printf("Correct answer was: %d\n", correct);
            break;
        }

        printf("✅ Correct!\n\n");
        score++;
        level++;
    }

    printf("\n=== GAME OVER ===\n");
    printf("Final Level: %d\n", level);
```



# Tic Tac Toe

SLIDE 08

SMART CPU

```c
/* ==================== TIC TAC TOE (SMART CPU) ==================== */

/* Checks board for a winner
   Returns:
   +10 if CPU (O) wins
   -10 if Player (X) wins
    0 otherwise */
int check_winner(char board[]) {
    int win[8][3] = {
        {0,1,2},{3,4,5},{6,7,8},  // rows
        {0,3,6},{1,4,7},{2,5,8},  // columns
        {0,4,8},{2,4,6}           // diagonals
    };

    for (int i = 0; i < 8; i++) {
        if (board[win[i][0]] == board[win[i][1]] &&
            board[win[i][1]] == board[win[i][2]]) {

            if (board[win[i][0]] == 'X') return -10;
            if (board[win[i][0]] == 'O') return 10;
        }
    }
    return 0;
}

/* Returns 1 if empty cells remain, 0 otherwise */
int is_moves_left(char board[]) {
    for (int i = 0; i < 9; i++)
        if (board[i] != 'X' && board[i] != 'O')
            return 1;
    return 0;
}

/* Minimax algorithm
   Recursively evaluates all possible future states */
int minimax(char board[], int depth, int isMax) {

    int score = check_winner(board);

    /* Base cases */
    if (score == 10) return score - depth;
    if (score == -10) return score + depth;
    if (!is_moves_left(board)) return 0;

    /* CPU's turn (maximize score) */
    if (isMax) {
        int best = -1000;

        for (int i = 0; i < 9; i++) {
            if (board[i] != 'X' && board[i] != 'O') {
                char temp = board[i];
                board[i] = 'O';

                int val = minimax(board, depth + 1, 0);
                best = (best > val) ? best : val;

                board[i] = temp;
            }
        }
        return best;
    }
    /* Player's turn (minimize score) */
    else {
        int best = 1000;

        for (int i = 0; i < 9; i++) {
            if (board[i] != 'X' && board[i] != 'O') {
                char temp = board[i];
                board[i] = 'X';

                int val = minimax(board, depth + 1, 1);
                best = (best < val) ? best : val;

                board[i] = temp;
            }
        }
        return best;
    }
}

/* Determines the optimal CPU move */
int best_move(char board[]) {
    int bestVal = -1000;
    int move = -1;

    for (int i = 0; i < 9; i++) {
        if (board[i] != 'X' && board[i] != 'O') {
            char temp = board[i];
            board[i] = 'O';

            int moveVal = minimax(board, 0, 0);
            board[i] = temp;

            if (moveVal > bestVal) {
                bestVal = moveVal;
                move = i;
            }
        }
    }
    return move;
}

/* Prints the Tic Tac Toe board */
void print_board(char board[]) {
    printf("\n");
    for (int i = 0; i < 9; i++) {
        printf(" %c ", board[i]);
        if (i % 3 != 2) printf("|");
        if (i == 2 || i == 5) printf("\n-----------\n");
    }
    printf("\n");
}
```



© 2025

# Tic Tac Toe

CONT. PG 2

SLIDE 08b

```c
/* Main Tic Tac Toe game loop */
void tic_tac_toe() {
    char board[9] = {'1','2','3','4','5','6','7','8','9'};
    int move;

    printf("\n--- TIC TAC TOE (SMART CPU) ---\n");
    printf("You are X | CPU is O\n");

    for (int turn = 0; turn < 9; turn++) {

        print_board(board);

        /* Player move */
        if (turn % 2 == 0) {
            printf("Your move: ");
            scanf("%d", &move);
            move--;

            if (move < 0 || move > 8 || board[move] == 'X' || board[move] == 'O') {
                printf("Invalid move!\n");
                turn--;
                continue;
            }
            board[move] = 'X';
        }
        /* CPU move */
        else {
            printf("CPU is thinking...\n");
            move = best_move(board);
            board[move] = 'O';
        }

        int result = check_winner(board);
        if (result == 10) {
            print_board(board);
            printf("CPU wins 🤖\n");
            return;
        }
        if (result == -10) {
            print_board(board);
            printf("You win 🎉\n");
            return;
        }
    }

    print_board(board);
    printf("It's a draw 🤝\n");
}
```



© 2025

# Conclusion

FINAL SLIDE

What We Learned Building ARCADE VAULT in C

### ♠ Card Bet Game

- rand() & srand() for random card generation
- Modulo (%) to check Odd / Even outcomes
- while loop for continuous game rounds
- Betting logic with coin balance tracking
- Input validation with if/else conditions

### ▯ Wordle Game

- String comparison using strcmp()
- strlen() for input length validation
- 2-pass feedback algorithm (green/yellow)
- Array tracking for letter usage (used[])
- Loop control with continue statement

### × Tic Tac Toe

- 2D array logic for 3×3 board state
- Minimax algorithm for unbeatable AI
- Recursive functions with depth tracking
- Best move evaluation using scoring
- Turn-based loop with win detection

### ≡ Math Survival

- Time library: time() and difftime()
- Dynamic difficulty scaling by level
- Integer division for clean ÷ questions
- Score and level tracking variables
- Game loop with timeout detection

**CORE C CONCEPTS USED:**

Functions & Prototypes | Loops: while / for / do-while | switch/case Menu Navigation | scanf / printf I/O Handling | Arrays & Strings



THANK YOU!

# THANK YOU!

for watching our presentation

## ARCADE VAULT

A Multi-Game Platform Built in C


PRESENTED BY

01. NOEL ROBIN NINAN
02. ADITYA GOYAL
03. ARNAV LOKHANDE
04. RITESH POOJARI


"From loops to logic — we built it all!"

Card Bet · Wordle · Tic Tac Toe · Math Survival



© 2025